ColdChain AI — Viva Q&A

Every question Professor Sharma asked during 3 review rounds,
and how we fixed it / how to answer it.
Team: Hiren Patel (68), Zeeshan Ansari (70) | **Subject:** Data Science-I (Term Work)

Part 1 — Round 1: 13 errors found and fixed

Error 1: Shuffled train/test split on time-series data

Problem: We used train_test_split (random shuffle) on hourly sensor data. The model trained on future hours and was tested on past hours — scores looked better than reality.

Fix: Chronological split — first 80% of hours for training, last 20% for testing. Cross-validation uses TimeSeriesSplit (5 splits), so the model never trains on the future. New function *chrono_split()* in train.py.

Error 2: No naive baselines

Problem: Without a baseline we could not prove the models learned anything at all.

Fix: Added two baselines to metrics.csv — *persistence* (next hour = current hour, RMSE 1.154) for regression and *majority class* (always predict "normal", F1 0.0) for classification. Every model beats its baseline.

Error 3: power_bad and compressor_on correlated −1.0

Problem: compressor_on was exactly (1 − power_bad) — a perfect duplicate. It makes LinearRegression coefficients unstable and adds nothing.

Fix: Dropped compressor_on in features.py (with an explaining comment). Kept power_bad.

Error 4: No model-selection rule

Problem: Different metrics crowned different winners and we had no stated rule.

Fix: Rule written at the top of train.py before training: regression winner = lowest test RMSE; classification winner = highest test F1 (accuracy misleads on 14% minority class). Tie → simpler model wins.

Error 5: Unfair class-imbalance handling

Problem: 4 classifiers used class_weight="balanced" but KNN and NaiveBayes (no such option) got nothing — unfair leaderboard.

Fix: KNN and NaiveBayes train on randomly oversampled data (minority class duplicated to match majority). Oversampling happens *inside* each CV fold for KNN, never before the split, so no leakage.

Error 6: Missing evaluation artefacts

Problem: No confusion matrix, residual plot, or feature importance anywhere.

Fix: New src/plots.py generates results/confusion_matrix.png, results/residuals.png, results/feat_importance.png for the winning models.

Error 7: Bad hour/day encodings

Problem: Raw hour 0–23 tells KNN that 23:00 and 00:00 are far apart (they are neighbours). Plain day 0–6 is a meaningless ordinal.

Fix: Cyclic encoding — hour_sin and hour_cos (points on a 24-hour circle). Dropped the day column.

Error 8: Hyperparameter grids hit their edges

Problem: Best KNN k was the grid maximum — the true optimum might lie outside the grid.

Fix: Widened grids (KNN k up to 35, tree depths up to 12/15/None). Winners now sit inside their grids (tree depth 3 of 3–None — see Q&A).

Error 9: One-sided excursion class

Problem: All excursions were over-heating (>8°C). The "excursion detector" had never seen a freeze.

Fix: Added rare thermostat-fault events (cooling stuck ON for several hours in a row) to make_data.py. Data now has 373 heat + 32 freeze excursions.

Errors 10–13: Small hygiene issues

Problem: README said 90 days but code did 120; README promised docs/ files that didn't exist; "Zeeshan Anasari" typo; GridSearchCV with empty grid = fake tuning.

Fix: README corrected to 120 days; docs/ now has experiments.md + 3 review reports + this Q&A; name fixed to Zeeshan Ansari; LinearRegression and NaiveBayes are fit directly with an honest comment.

Part 2 — Round 1: 12 silly questions (30-second answers)

1. What does random_state=42 do?

Answer: It fixes all randomness (data split, tree building, CV shuffling) so results are reproducible. Change it to 0 → slightly different split and scores. Delete it → different results on every run.

2. Why scale for KNN/SVM but not for trees?

Answer: KNN and SVM use distances, so they are sensitive to feature scales. Trees only compare "feature > threshold", so scale is irrelevant. Scaling trees breaks nothing; it is just unnecessary.

3. corr(power_bad, compressor_on) = −1.0 exactly. Problem or good news?

Answer: A problem — perfect multicollinearity. One column is exactly 1 minus the other, so it adds zero information and makes LinearRegression coefficients unstable. We dropped compressor_on.

4. KNN accuracy 0.79 vs majority baseline 0.87 — did KNN learn anything?

Answer: Yes. Accuracy misleads here: the baseline gets 0.87 by never predicting an excursion (F1 = 0.0). KNN's F1 is 0.42 — it actually finds excursions, which is the whole point of the project.

5. You ran GridSearchCV on LinearRegression with an empty grid {}. What was searched?

Answer: Nothing — that was tuning theatre. An empty grid just re-fits defaults 5 times. We removed it; LinearRegression is now fit directly with a comment saying why.

6. KNN picked k=35, the grid maximum. How do you know 50 isn't better?

Answer: Honestly we don't — the optimum could be outside. But KNN is not our champion (F1 0.42 vs tree's 0.52), so extending the grid further would not change the winner. Noted as future work.

7. Why stratify the classification split but not the regression split?

Answer: stratify keeps the class ratio identical in train and test — it only works on discrete labels. The regression target (temperature) is continuous, so there is nothing to stratify.

8. Why 5-fold CV instead of a single validation split?

Answer: One split's score depends on luck — which hours landed in validation. Averaging 5 folds gives a stable, trustworthy estimate of the hyperparameter quality.

9. GridSearchCV vs RandomizedSearchCV — and why random only for RandomForest?

Answer: Grid tries every combination (thorough but slow); random samples n_iter combinations (faster, good enough). RandomForest had the biggest grid (3 params), so full grid search was too slow — we used RandomizedSearchCV with 10 iterations there.

10. rolling(3).mean() at row t — does it peek at the future?

Answer: No. Pandas rolling is backward-looking: row t averages rows t−2, t−1, t. The future (t+1) is never used.

11. precision_score with no average argument — what does it compute?

Answer: For binary labels it defaults to the positive class (label 1 = excursion): of all hours we flagged as excursions, how many truly were.

12. Your day column started at t=0. Which weekday was it?

Answer: Unknowable — t=0 is arbitrary, so the 0–6 labels were meaningless ordinals. That is exactly why we dropped the day column in round 1.

Part 3 — Round 1: 10 important questions

1. Why is shuffling time-series data wrong?

Answer: Three reasons. (a) Autocorrelation: neighbouring hours look alike, so a shuffled test set is too easy. (b) Future leakage: the model trains on hours that occur after the test hours — impossible in deployment. (c) Deployment reality: you always train on the past and predict the future. The fix is a chronological split plus TimeSeriesSplit CV.

2. Which single model do you ship to guard medicines worth lakhs — and why not the accuracy winner?

Answer: The DecisionTree (F1 0.52). Accuracy is the wrong criterion: always predicting "normal" already scores 0.87 accuracy with F1 0.0. A false negative (missed excursion → spoiled medicines) costs lakhs; a false positive (false alarm → someone checks the room) costs minutes. We optimize F1/recall, not accuracy.

3. Name two other ways to handle class imbalance, with trade-offs.

Answer: (a) SMOTE — creates synthetic minority samples instead of duplicating; richer than plain oversampling but can blur class boundaries and is outside our syllabus. (b) Threshold moving — predict "excursion" when P > 0.3 instead of 0.5; boosts recall at the cost of precision, and needs no retraining. We used class_weight + plain oversampling as the syllabus-faithful options.

4. Best RMSE 1.0374 vs persistence 1.154 — only ~10% better. What does that say?

Answer: The problem is genuinely hard: next hour's temperature depends on unpredictable shocks (power cuts, long door openings) that no current-hour feature can foresee. Our features capture thermal inertia and current conditions well (R^2 0.5149), but the unpredictable part stays unpredictable. The 10% is honest, not a failure.

5. Why do lag features help? Connect to the physics.

Answer: A cold room has thermal inertia — it warms/cools slowly. Our simulator's *0.60* × *temp[i−1]* term is exactly that: 60% of this hour's temp carries over. temp_lag1, temp_lag2 and temp_roll3 let the model see that inertia.

6. Explain the bias–variance trade-off in your grids.

Answer: Shallow trees (depth 3) have high bias — too simple, they underfit (R^2 0.45). Deep single trees have high variance — they memorize noise. RandomForest averages many trees, cutting variance while keeping bias low, which is why the forest beats the single tree at the same depth. The forest's best depth stayed shallow (5) because the data is noisy — deeper trees just fit noise.

7. (Round 1) All excursions were heat events. What couldn't the classifier do?

Answer: It had never seen a freeze, so a sub-2°C event would likely be misclassified as normal — a dangerous blind spot for vaccines. Fixed in round 2: thermostat-fault events now create 32 freeze excursions, so the classifier sees both types.

8. Why does raw hour 0–23 mislead KNN/SVM?

Answer: Geometrically, 23:00 and 00:00 are 1 hour apart on a clock but 23 units apart as integers — distance-based models (KNN, SVM) compute nonsense distances. hour_sin/hour_cos place the hours on a circle where neighbours stay neighbours.

9. The facility replaces its compressor with a stronger one. What happens?

Answer: Concept drift — the temperature dynamics change, so the trained model's error rises over the coming weeks. Detect it by monitoring live prediction error (or the share of flagged excursions) against a threshold; retrain monthly, or immediately when drift is detected, on fresh data.

10. Three things that break with real sensor data, and the syllabus fix for each?

Answer: (a) Missing readings → imputation (mean/median or forward-fill). (b) Sensor spikes/drift → outlier detection and clipping, recalibration checks. (c) Unlogged door events → nothing in the syllabus can invent missing data; document it as a limitation and add a door sensor.

Part 4 — Round 2: 5 new errors found and fixed

N1 (serious): Regression baseline used the wrong lag

Problem: The persistence baseline used temp_lag1 (temperature 2 hours ago) against next hour's temperature — a 2-hour gap, not persistence. Reported RMSE 1.51 flattered every model.

Fix: Baseline now uses Xte["temp"] (current hour) → true 1-hour persistence, RMSE 1.154. The professor recomputed it independently and matched. Story is now honest: LinearRegression 1.0374 vs 1.154, ~10% better.

N2 (serious): Oversampling before TimeSeriesSplit leaked across folds

Problem: Duplicated minority rows were split across CV train/validation folds, so KNN's tuning scores were optimistic.

Fix: New tune_knn() loop oversamples only inside each fold's training indices; validation folds stay untouched. NaiveBayes is fit directly on oversampled train with no fake CV. (Side proof it mattered: KNN's best k moved 3 → 35 after the fix.)

N3: plots.py could silently mislabel a future winner

Problem: The regressor plot branch fell back to LinearRegression for any unhandled winner.

Fix: Both branches now raise SystemExit asking for a builder instead of mislabeling.

N4: README went stale

Problem: False "every model is tuned" claim, champions unnamed, seaborn listed but unused.

Fix: README now exempts LinearRegression/NaiveBayes, names both champions (Linear Regression, Decision Tree), and the install line is pandas + scikit-learn + matplotlib only.

N5: Tie-break rule was undocumented

Problem: DecisionTree and RandomForest tied at F1 0.5152; the tree was crowned with no written rule.

Fix: "Simpler model wins ties" is now documented in train.py comments, experiments.md, and applied consistently (plots.py agrees).

Part 5 — Round 3: the 8 final viva questions (PASS)

1. 120 × 24 = 2880 rows, but your CSV has 2879. Where did one row go?

Answer: The target temp_next = temp.shift(−1) needs a "next hour", so the final row has no target and is dropped → 2879. Feature engineering then drops 2 more rows for the lag features → 2877. No data lost — both drops are expected.

2. What are hour_sin and hour_cos? Why not hour = 0..23?

Answer: Cyclic encoding of time of day: sin(2π·hour/24), cos(2π·hour/24). Raw integers tell distance-based models that 23:00 and 00:00 are 23 apart, when they are 1 hour apart. On the circle, neighbours stay neighbours.

3. Two raw features correlated exactly −1.0 — wonderful or a problem?

Answer: A problem: perfect multicollinearity. compressor_on was exactly 1 − power_bad, added zero information and destabilized linear-model coefficients. We dropped compressor_on.

4. Barely 10% better than persistence — expensive thermometer?

Answer: It learned real structure: R^2 0.52, and it beats persistence on every metric, consistently across 5 time-series folds. The 10% gap is the honest ceiling — next hour's temperature depends on unpredictable shocks (power cuts, door openings) that no current reading can foresee.

5. Did you oversample the test set? What if you had?

Answer: No — only the training data (and inside CV folds). Oversampling the test set would fake the scores upward because the test set must reflect real-world class ratios. That would be cheating the evaluation.

6. Tree and forest tied at F1 0.5152 — why crown the tree?

Answer: Our tie-break rule, written before training: the simpler model wins ties. One tree vs one hundred: the tree trains faster, needs less memory, and is explainable in a viva — with identical F1, there is no reason to pay for the forest.

7. Recall is 0.46 — you miss more than half the excursions. Ship it Monday?

Answer: Not for medicines. First change: lower the decision threshold (flag "excursion" at P > 0.3 instead of 0.5) to buy recall with extra false alarms — a false alarm costs a room check, a missed excursion costs spoiled vaccines. Second: collect more excursion examples; 405 is thin.

8. Winning tree depth is 3 — the grid minimum. How do you know 2 wouldn't win?

Answer: Honestly, we don't — depth 2 was never tested, so the optimum could sit outside the grid. The selection among tested values was fair and the winner beats the baseline; extending the grid downward is listed as future work.

Final results (honest numbers): Regression — Linear Regression, RMSE 1.0374, R^2 0.5149 (baseline persistence 1.154). Classification — Decision Tree, accuracy 0.889, F1 0.515 (baseline majority F1 0.0). 120 days hourly data, 2877 rows, 14% excursions (373 heat + 32 freeze). All models within the Data Science-I syllabus. Professor Sharma: PASS after 3 rounds.